

COMPUTER ORGANIZATION

NAME :ARITHMETIC

Outline

- Numbers - Arithmetic Operations and Characters
- Addition and Subtraction of Signed Numbers
- Design of Fast Adders
- Multiplication of Positive Numbers
- Signed Operand Multiplication
- Fast Multiplication
- Integer Division
- Floating-point Numbers and Operations

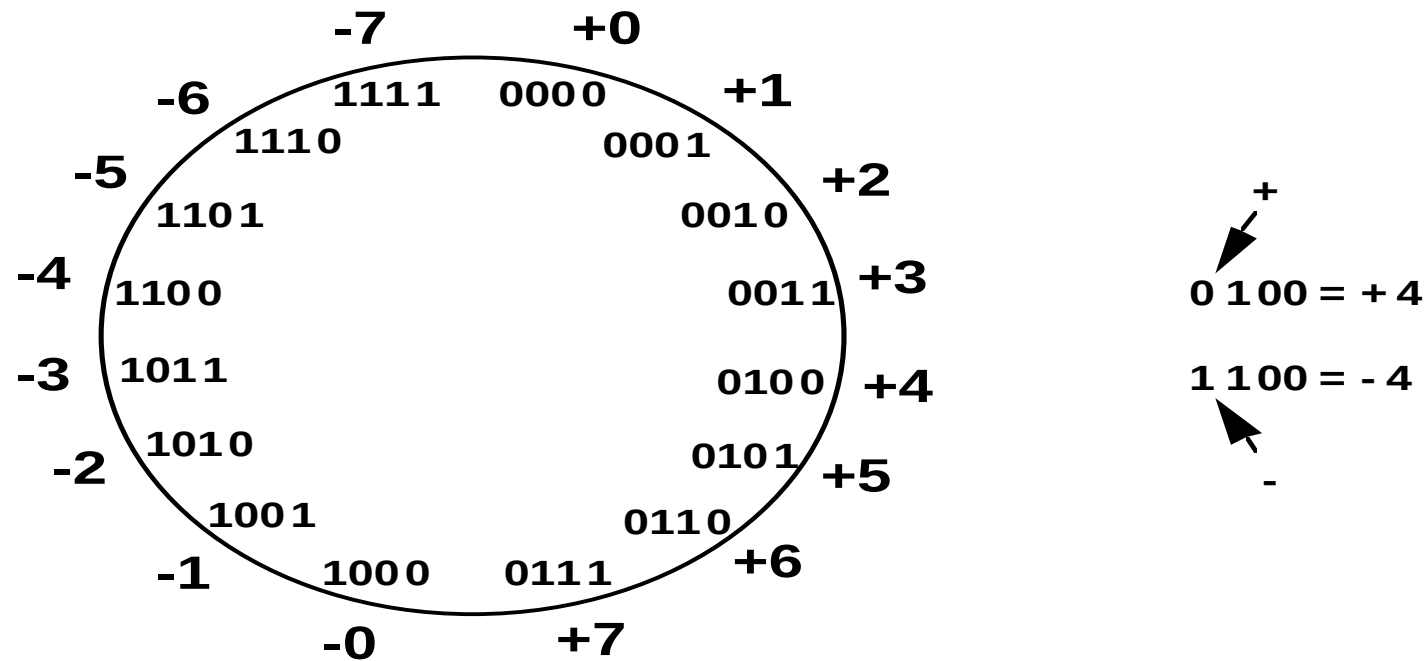
Learning Objectives

1. To understand number representation and addition/ subtraction in the 2s complement form.
2. To analyze the booth algorithm used to determine how multiplicand summands are selected by the multiplier bit patterns.
3. To analyze high speed adders using carry lookahead logic to generate carry signal in parallel.
4. To gain extensive knowledge in representing floating point numbers in the IEEE standard format and perform basic arithmetic operation.

Number, Arithmetic Operations, and Characters

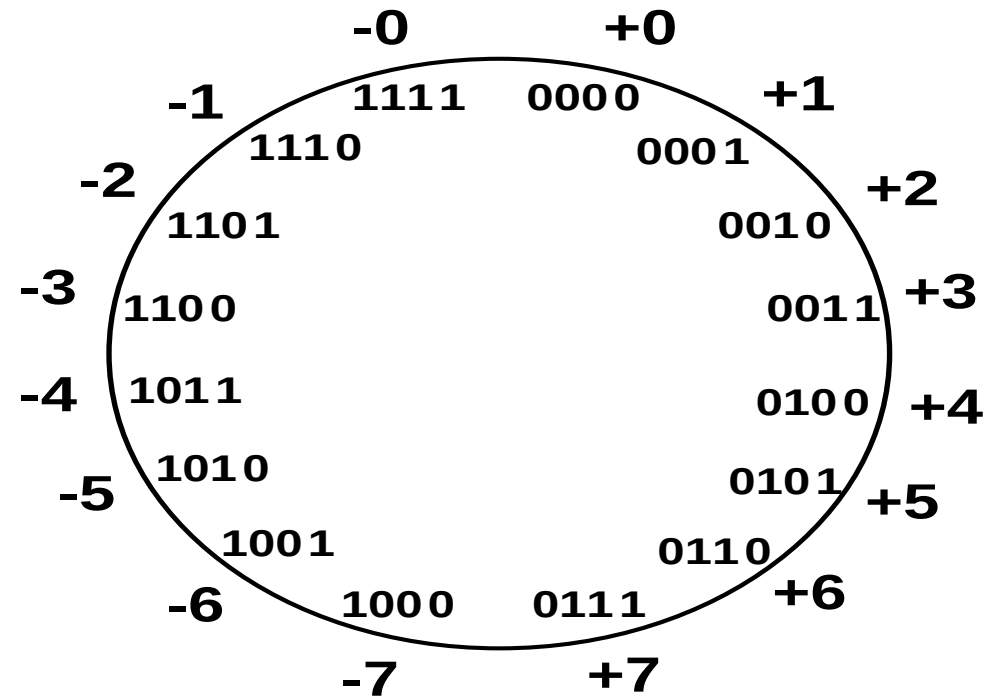
- Signed Integer
- 3 major representations:
 - Sign and magnitude
 - One's complement
 - Two's complement
- Assumptions:
 - 4-bit machine word
 - 16 different values can be represented
 - Roughly half are positive, half are negative

Sign and Magnitude Representation



High order bit is sign: 0 = positive (or zero), 1 = negative
Three low order bits is the magnitude: 0 (000) thru 7 (111)
Number range for n bits = $\pm 2^{n-1} - 1$
Two representations for 0

One's Complement Representation



$\begin{array}{c} + \\ \swarrow \\ 0\ 100 = +4 \\ \searrow \\ 1\ 011 = -4 \\ \swarrow \\ - \end{array}$

- Subtraction implemented by addition & 1's complement
- Still two representations of 0! This causes some problems
- Some complexities in addition

- $+3 \ \& \ -13 = 0110$
- $-14 \ \& \ -2 = 10000$
- $-7 \ \& \ +3 = 1100$
- $-13 \ \& \ 8 = 1011$
- $-24 \ \text{and} \ -8 = 0100001$
- $-36 \ \text{and} \ -9 = 0101110$
- $-9 \ \text{and} \ -4 = 0011$
- $-8 \ \text{and} \ +4 = 1100$

- -4 and +2

0010(+2)

0100(+4)=1011(1's complement)

1011

1

1100(-4)

1100

0010

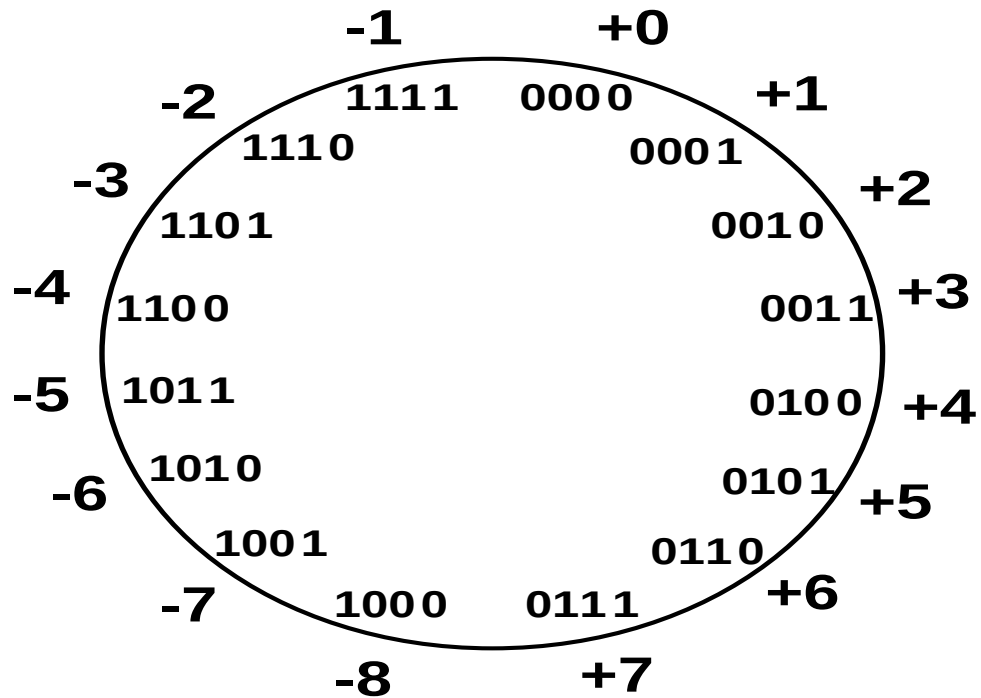
1110(-2) 0010=1101=1101

1

1110

Two's Complement Representation

*like 1's comp
except shifted
one position
clockwise*



$\begin{array}{c} + \\ \swarrow \\ 0\ 1\ 0\ 0 = +4 \\ \nwarrow \\ 1\ 1\ 0\ 0 = -4 \\ - \end{array}$

- Only one representation for 0
- One more negative number than positive number

Binary, Signed-Integer Representations

B				V alues represented		
b_3	b_2	b_1	b_0	Sign and magnitude	1' s complement	2' s complement
0	1	1	1	+ 7	+ 7	+ 7
0	1	1	0	+ 6	+ 6	+ 6
0	1	0	1	+ 5	+ 5	+ 5
0	1	0	0	+ 4	+ 4	+ 4
0	0	1	1	+ 3	+ 3	+ 3
0	0	1	0	+ 2	+ 2	+ 2
0	0	0	1	+ 1	+ 1	+ 1
0	0	0	0	+ 0	+ 0	+ 0
1	0	0	0	- 0	- 7	- 8
1	0	0	1	- 1	- 6	- 7
1	0	1	0	- 2	- 5	- 6
1	0	1	1	- 3	- 4	- 5
1	1	0	0	- 4	- 3	- 4
1	1	0	1	- 5	- 2	- 3
1	1	1	0	- 6	- 1	- 2
1	1	1	1	- 7	- 0	- 1

Figure 2.1. Binary, signed-integer representations.

Addition and Subtraction – 2's Complement

If carry-in to the high order bit = carry-out then ignore carry

if carry-in differs from carry-out then overflow

$$\begin{array}{r}
 4 \quad 0100 \\
 + 3 \quad 0011 \\
 \hline
 7 \quad 0111
 \end{array}$$

$$\begin{array}{r}
 -4 \quad 1100 \\
 + (-3) \quad 1101 \\
 \hline
 -7 \quad 11001
 \end{array}$$

$$\begin{array}{r}
 4 \quad 0100 \\
 - 3 \quad 1101 \\
 \hline
 1 \quad 10001
 \end{array}$$

$$\begin{array}{r}
 -4 \quad 1100 \\
 + 3 \quad 0011 \\
 \hline
 -1 \quad 1111
 \end{array}$$

Simpler addition scheme makes twos complement the most common choice for integer number systems within digital systems

Addition/subtraction of signed numbers

x_i	y_i	Carry-in c_i	Sum s_i	Carry-out c_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

At the i^{th} stage:

Input:

c_i is the carry-in

Output:

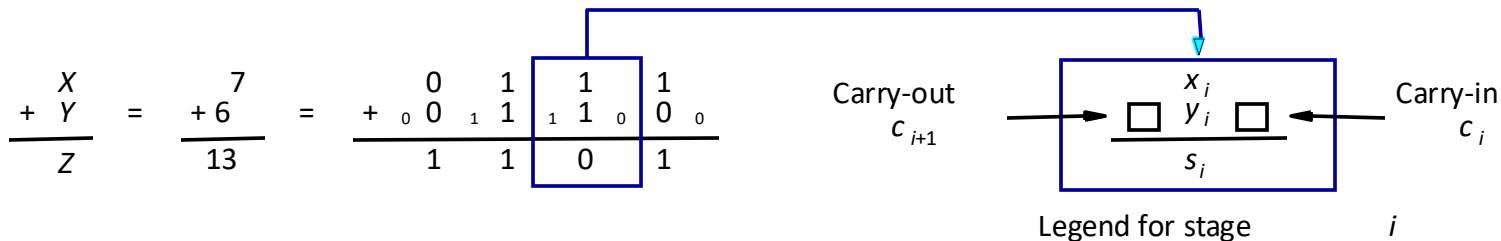
s_i is the sum

c_{i+1} carry-out to $(i+1)^{st}$ state

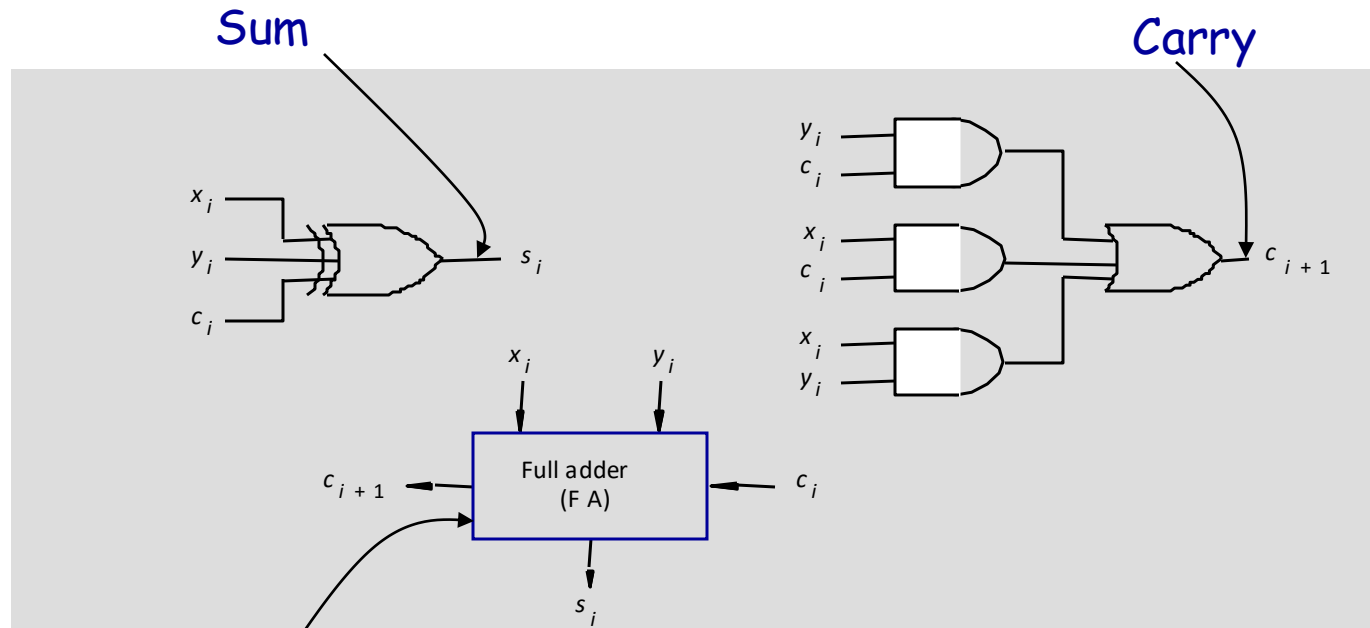
$$s_i = \overline{x_i} \overline{y_i} c_i + \overline{x_i} y_i \overline{c_i} + x_i \overline{y_i} \overline{c_i} + x_i y_i c_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = y_i c_i + x_i c_i + x_i y_i$$

Example:



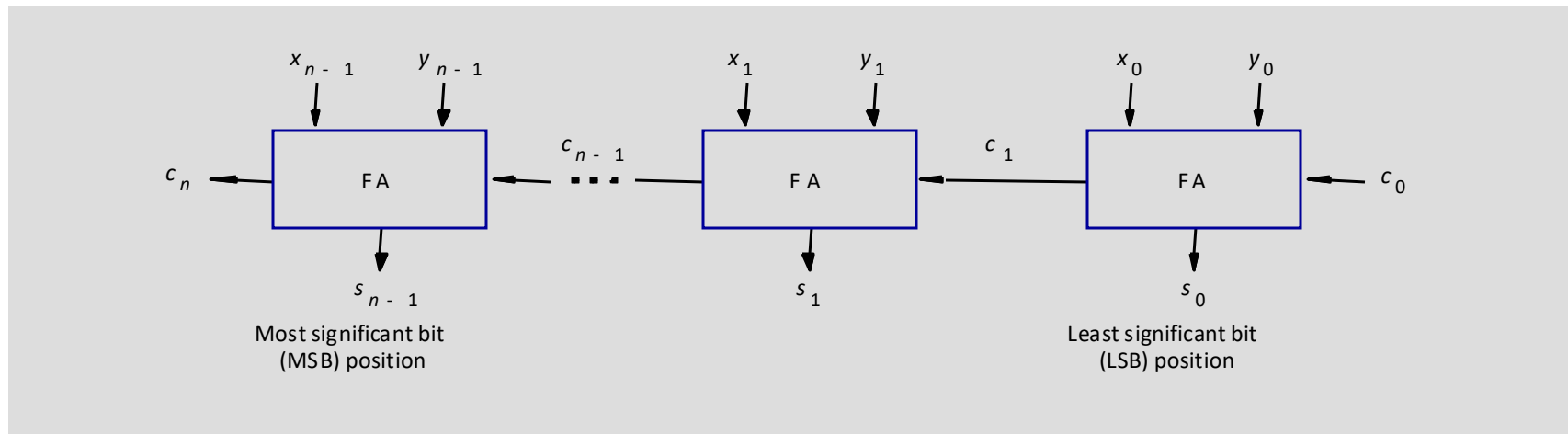
Addition logic for a single stage



Full Adder (FA): Symbol for the complete circuit for a single stage of addition.

n -bit adder

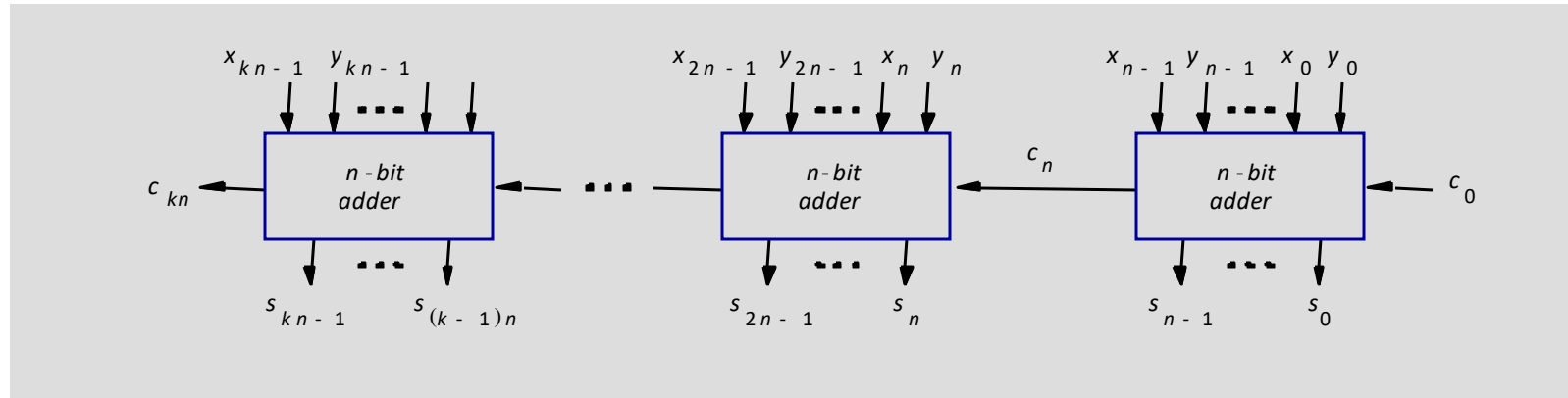
- Cascade n full adder (FA) blocks to form a n -bit adder.
- Carries propagate or ripple through this cascade, [n-bit ripple carry adder](#).



Carry-in c_0 into the LSB position provides a convenient way to perform subtraction.

K n -bit adder

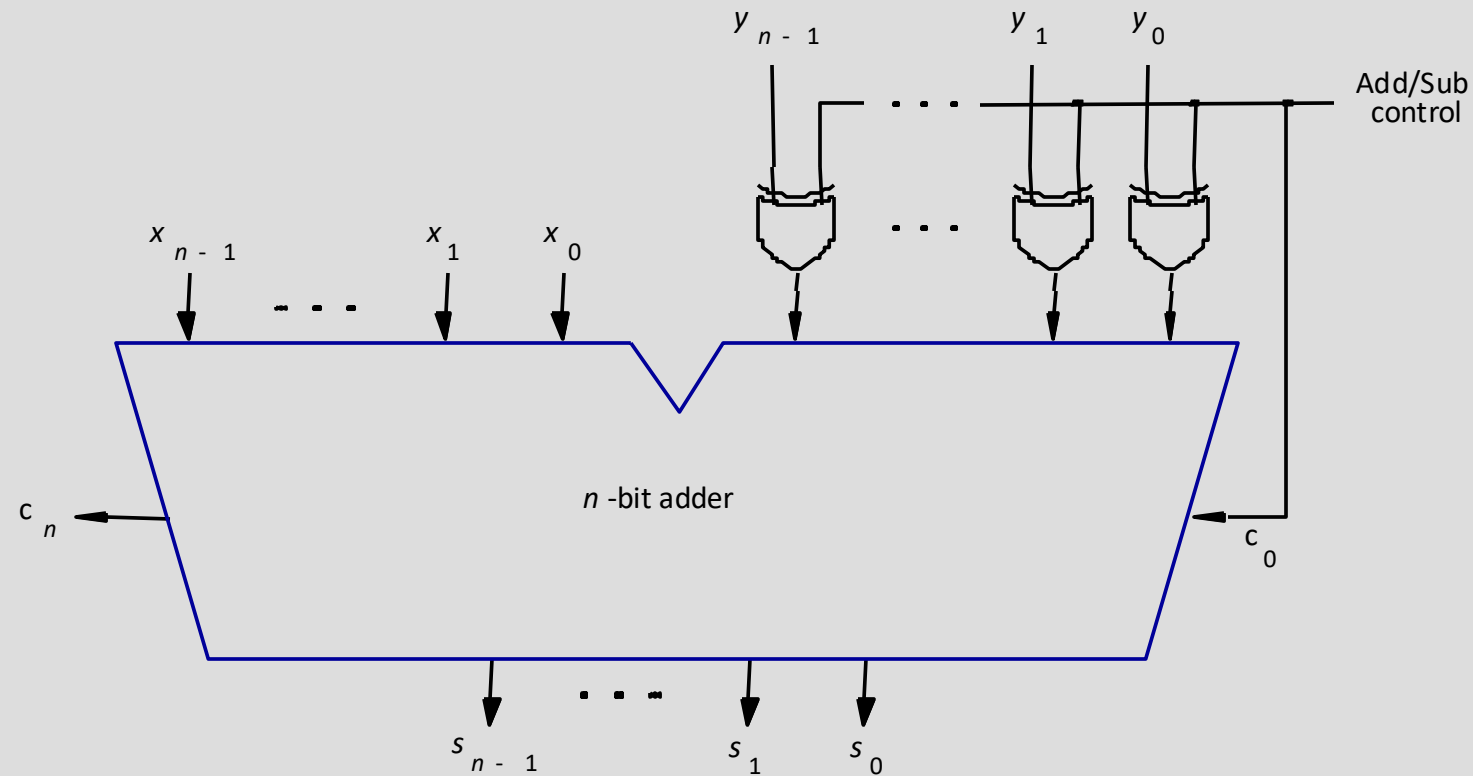
K n -bit numbers can be added by cascading k n -bit adders.



Each n -bit adder forms a block, so this is cascading of blocks.

Carries ripple or propagate through blocks, [Blocked Ripple Carry Adder](#)

n -bit adder/subtractor (contd..)



- Add/sub control = 0, addition.
- Add/sub control = 1, subtraction.

Recall the equations:

$$S_i = x_i \oplus y_i \oplus c_i$$

$$C_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

Second equation can be written as:

$$C_{i+1} = x_i y_i + (x_i + y_i) c_i$$

We can write:

$$C_{i+1} = G_i + P_i c_i$$

$$\text{where } G_i = x_i y_i \text{ and } P_i = x_i + y_i$$

- G_i is called generate function and P_i is called propagate function
- G_i and P_i are computed only from x_i and y_i and not c_i , thus they can be computed in one gate delay after X and Y are applied to the inputs of an n -bit adder.

Carry lookahead

$$c_{i+1} = G_i + P_i c_i$$

$$c_i = G_{i-1} + P_{i-1} c_{i-1}$$

$$\Rightarrow c_{i+1} = G_i + P_i (G_{i-1} + P_{i-1} c_{i-1})$$

continuing

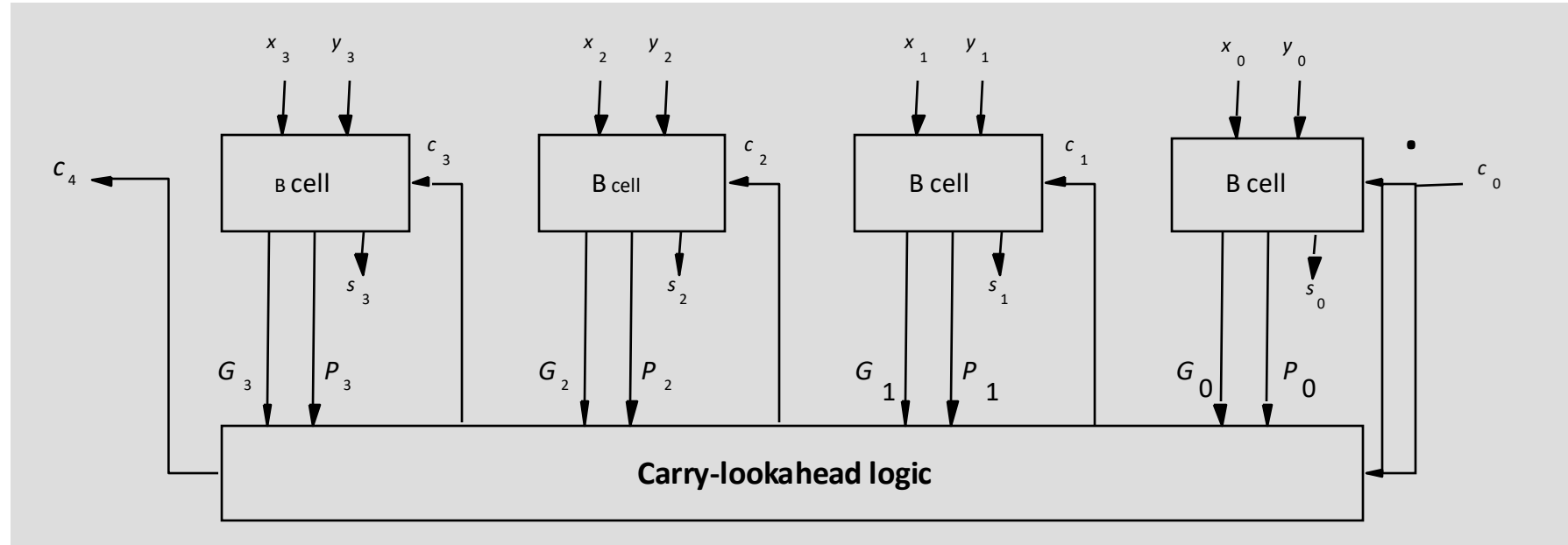
$$\Rightarrow c_{i+1} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} c_{i-2}))$$

until

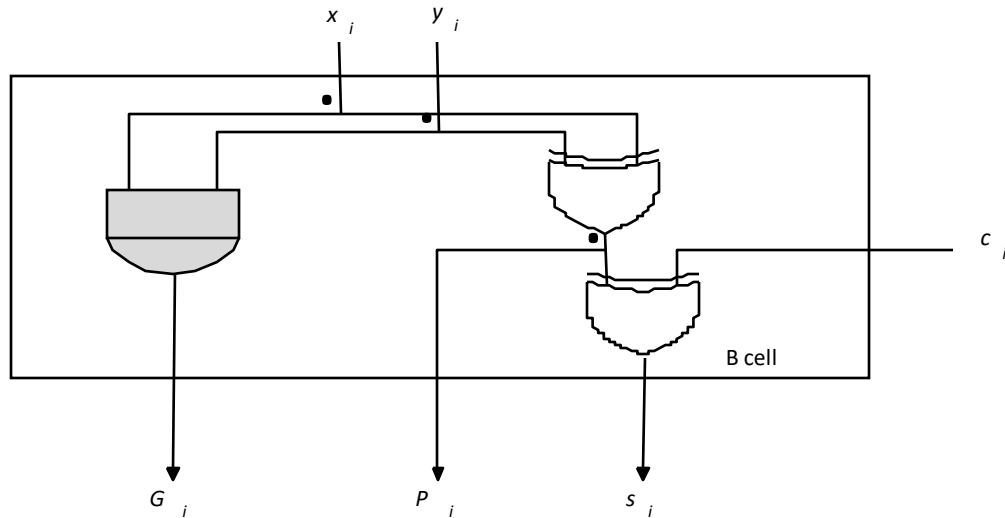
$$c_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \dots + P_i P_{i-1} \dots P_1 G_0 + P_i P_{i-1} \dots P_0 c_0$$

- All carries can be obtained 3 gate delays after X, Y and c_0 are applied.
 - One gate delay for P_i and G_i
 - Two gate delays in the AND-OR circuit for c_{i+1}
- All sums can be obtained 1 gate delay after the carries are computed.
- Independent of n , n -bit addition requires only 4 gate delays.
- This is called Carry Lookahead adder.

Carry-lookahead adder



**4-bit
carry-lookahead
adder**



B-cell for a single stage

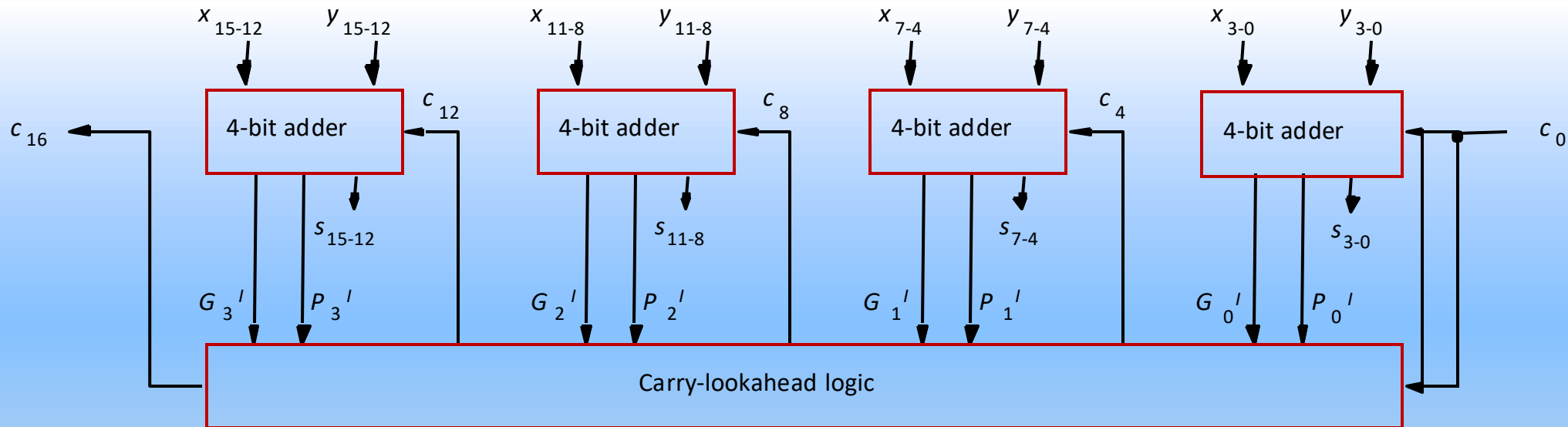
Carry lookahead adder (contd..)

- Performing n -bit addition in 4 gate delays independent of n is good only theoretically because of fan-in constraints.

$$c_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \dots + P_i P_{i-1} \dots P_1 G_0 + P_i P_{i-1} \dots P_0 c_0$$

- Last AND gate and OR gate require a fan-in of $(n+1)$ for a n -bit adder.
 - For a 4-bit adder ($n=4$) fan-in of 5 is required.
 - Practical limit for most gates.
- In order to add operands longer than 4 bits, we can cascade 4-bit Carry-Lookahead adders. Cascade of Carry-Lookahead adders is called Blocked Carry-Lookahead adder.

16 bit Carry-Lookahead adder



After x_i, y_i and c_0 are applied as inputs:

- G_i and P_i for each stage are available after 1 gate delay.
- P^i is available after 2 and G^i after 3 gate delays.
- All carries are available after 5 gate delays.
- c_{16} is available after 5 gate delays.
- s_{15} which depends on c_{12} is available after 8 (5+3) gate delays
(Recall that for a 4-bit carry lookahead adder, the last sum bit is available 3 gate delays after all inputs are available)

Multiplication

Multiplication of unsigned numbers

				1	1	0	1	(13) Multiplicand M
				1	0	1	1	(11) Multiplier Q
				1	1	0	1	
			1	1	0	1		
		0	0	0	0			
	1	1	0	1				
1	0	0	0	1	1	1	1	(143) Product P

Product of 2 n -bit numbers is at most a $2n$ -bit number.

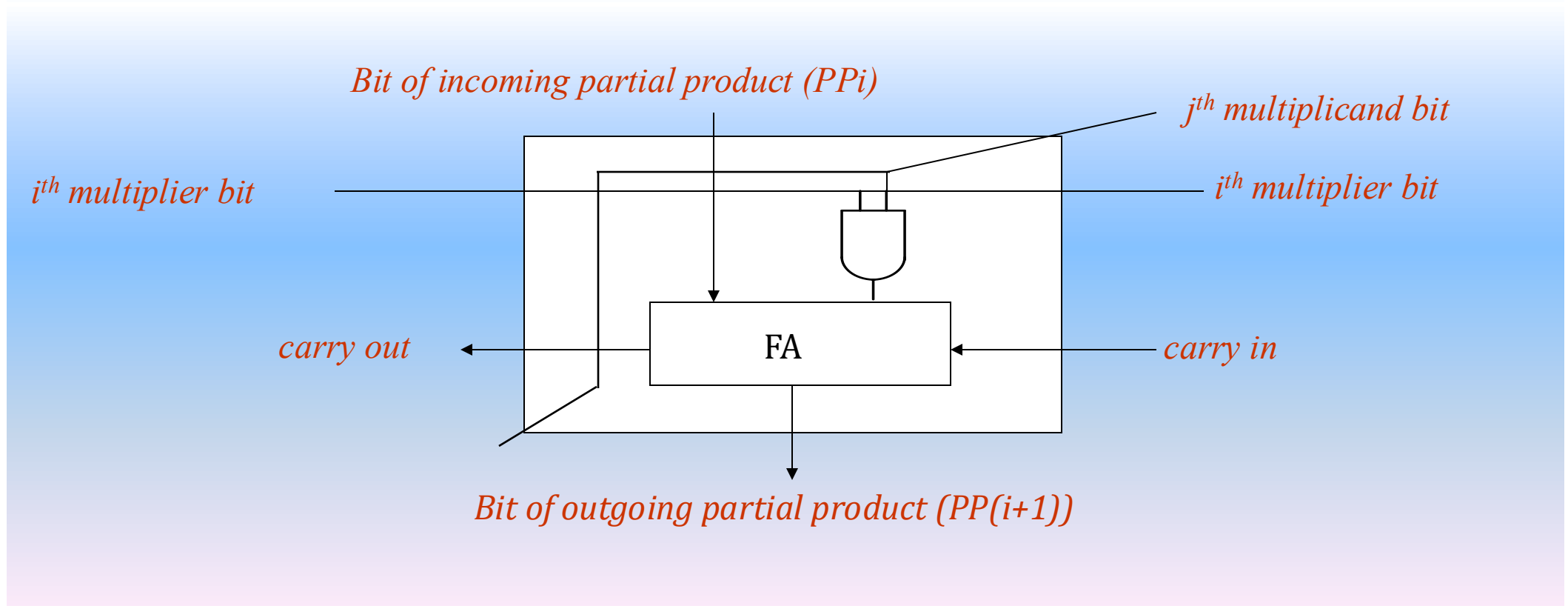
Unsigned multiplication can be viewed as addition of shifted versions of the multiplicand.

Multiplication of unsigned numbers (contd..)

- We added the partial products at end.
 - Alternative would be to add the partial products at each stage.
- Rules to implement multiplication are:
 - If the i^{th} bit of the multiplier is 1, shift the multiplicand and add the shifted multiplicand to the current value of the partial product.
 - Hand over the partial product to the next stage
 - Value of the partial product at the start stage is 0.

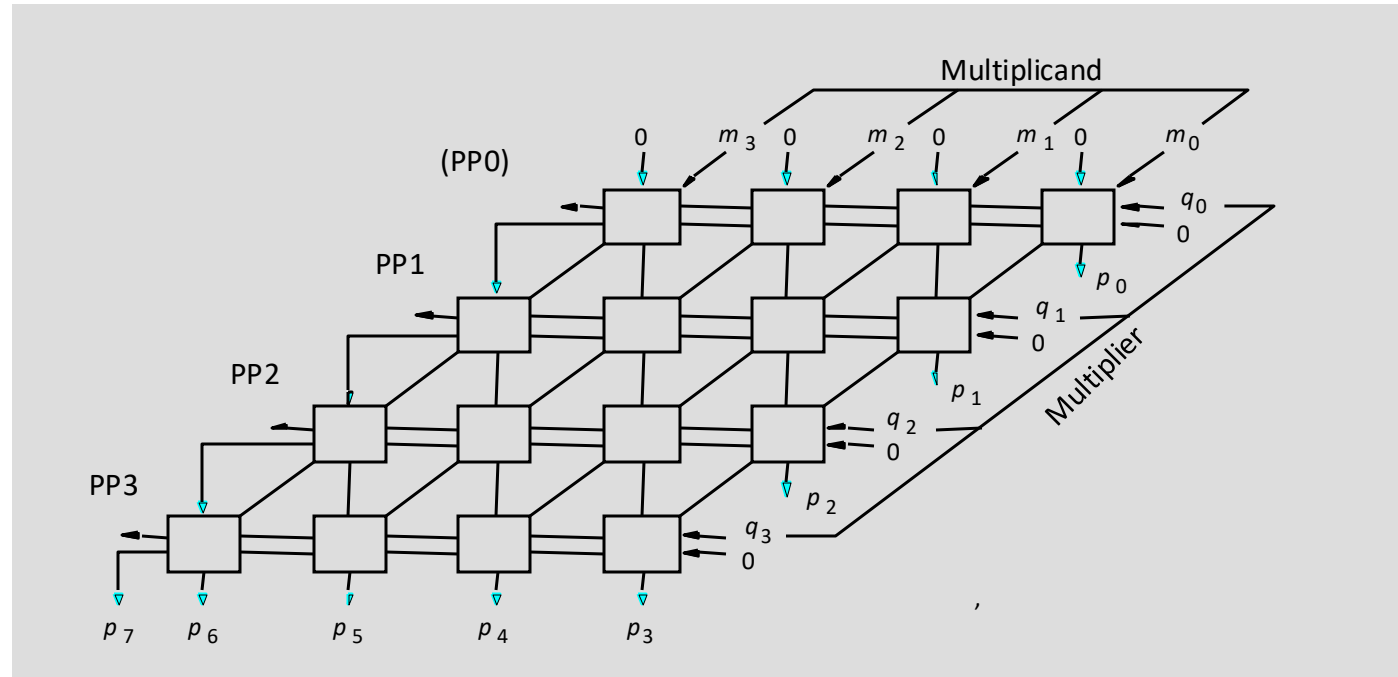
Multiplication of unsigned numbers

Typical multiplication cell



Combinatorial array multiplier

Combinatorial array multiplier



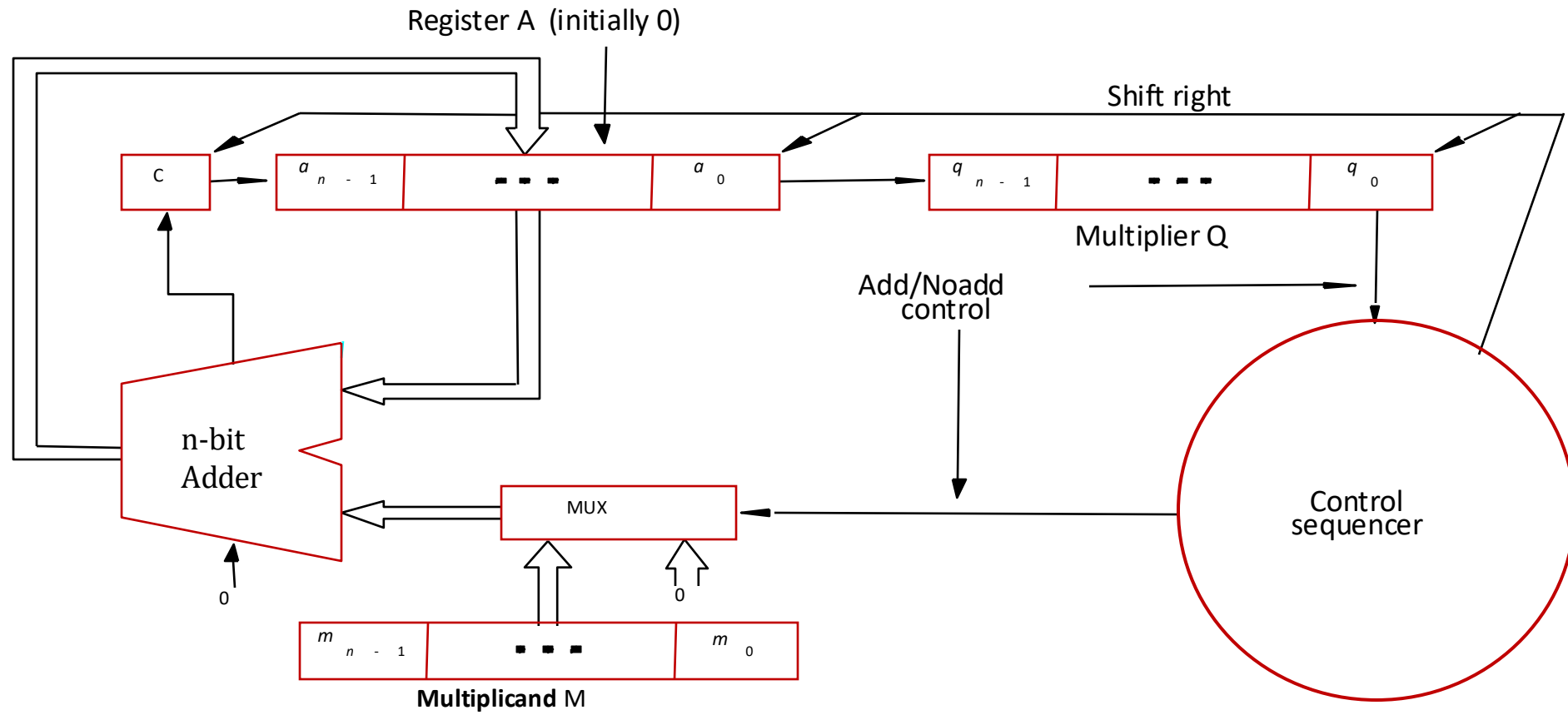
Product is: $p_7, p_6 \dots p_0$

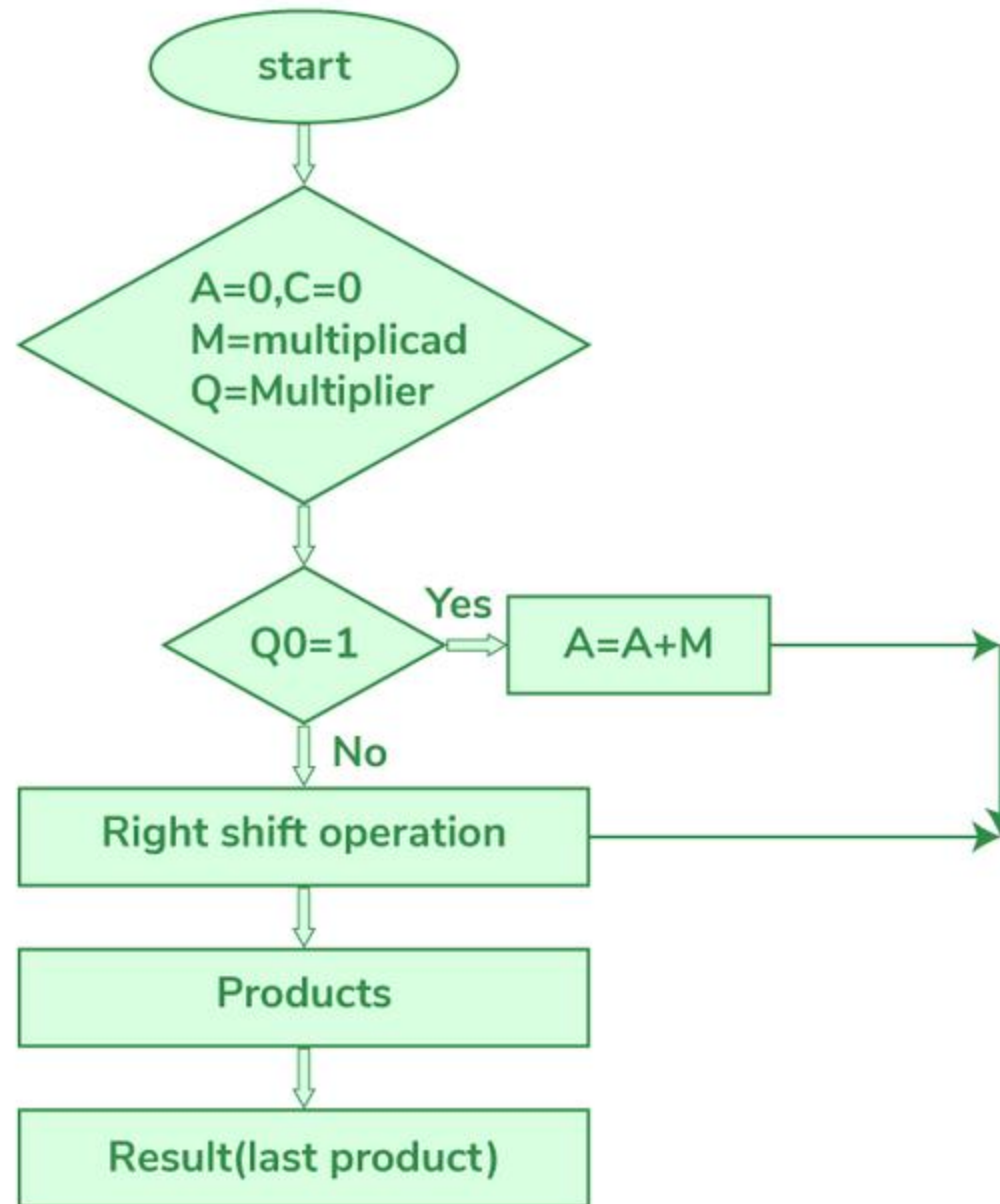
Multiplicand is shifted by displacing it through an array of adders.

Combinatorial array multiplier (contd..)

- Combinatorial array multipliers are:
 - Extremely inefficient.
 - Have a high gate count for multiplying numbers of practical size such as 32-bit or 64-bit numbers.
 - Perform only one function, namely, unsigned integer product.
- Improve gate efficiency by using a mixture of combinatorial array techniques and sequential techniques requiring less combinational logic.

Sequential Circuit Multiplier

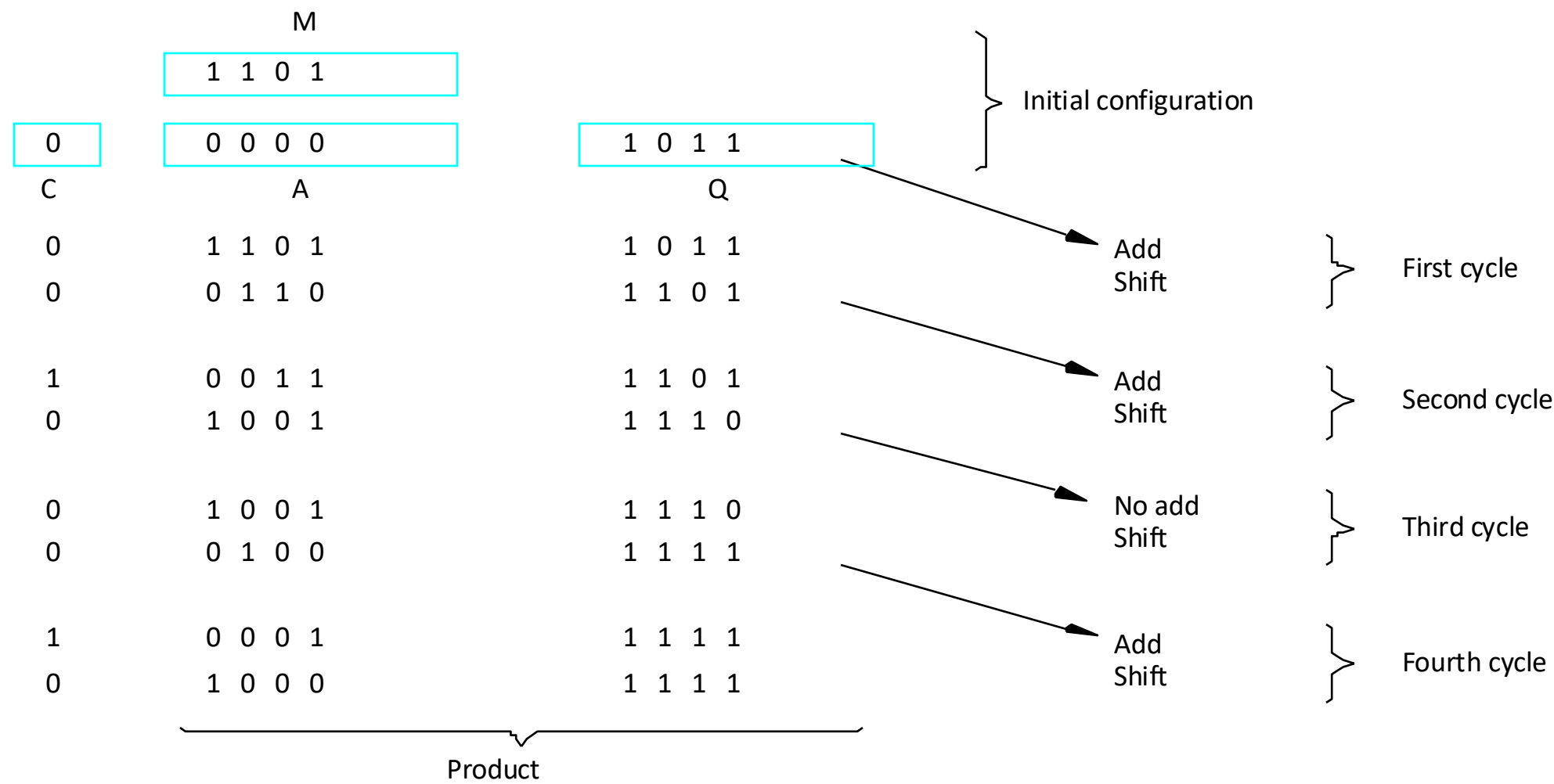




Sequential Multiplier

- **Initialization:**
 - Load multiplicand in “M” register, multiplier in “Q” register
 - Initialize “C” and “A” registers to all zeroes
- **Repeat the following steps “n” times, where “n” is the number of bits in the multiplier**
 - If (LSB of Q register == 1)
 - $A = A + M$ (carry-out goes to “C” register)
 - Treat the C, A and Q registers as one contiguous register and shift that register's contents right by one bit position
- **After the completion of “n” steps**
 - Register “A” contains high-order half of product
 - Register “Q” contains low-order half of product

Sequential multiplication (contd..)



Signed Multiplication

Signed Multiplication

- Considering 2's-complement signed operands, what will happen to $(-13) \times (+11)$ if following the same method of unsigned multiplication?

Sign extension is shown in blue

					1	0	0	1	1	(- 13)
					0	1	0	1	1	(+ 11)
	1	1	1	1	1	0	0	1	1	
	1	1	1	1	1	0	0	1	1	
	0	0	0	0	0	0	0	0		
	1	1	1	0	0	1	1			
	0	0	0	0	0	0				
	1	1	0	1	1	1	0	0	0	1 (- 143)

Signed Multiplication

- For a negative multiplier, a straightforward solution is to form the 2's-complement of both the multiplier and the multiplicand and proceed as in the case of a positive multiplier.
- This is possible because complementation of both operands does not change the value or the sign of the product.
- A technique that works equally well for both negative and positive multipliers – Booth algorithm.

Booth Algorithm

- Consider in a multiplication, the multiplier is positive 0011110, how many appropriately shifted versions of the multiplicand are added in a standard procedure?

									0	1	0	1	1	0	1
									0	0	+ 1	+ 1	+ 1	+ 1	0
										0	0	0	0	0	0
							0		1	0	1	1	0	1	
						0	1		0	1	1	0	1		
					0	1	0		1	1	0	1			
				0	1	0	1		1	0	1				
			0	0	0	0	0		0	0					
		0	0	0	0	0	0		0						
	0	0	0	0	0	0	0		0						
0	0	0	1	0	1	0	1	0	0	0	0	1	1	0	

Booth Algorithm

- Since $0011110 = 0100000 - 0000010$, if we use the expression to the right, what will happen?

							0	1	0	1	1	0	1		
							0	+	1	0	0	0	-	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0		
1	1	1	1	1	1	1	0	1	0	0	1	1		←	2's complement of the multiplicand
0	0	0	0	0	0	0	0	0	0	0	0				
0	0	0	0	0	0	0	0	0	0	0					
0	0	0	0	0	0	0	0	0	0						
0	0	0	0	0	0	0	0	0	0						
0	0	0	1	0	1	1	0	1							
0	0	0	0	0	0	0	0								
0	0	0	1	0	1	0	1	0	0	0	1	1	0		

Booth Algorithm

- In general, in the Booth scheme, -1 times the shifted multiplicand is selected when moving from 0 to 1, and +1 times the shifted multiplicand is selected when moving from 1 to 0, as the multiplier is scanned from right to left.

0	0	1	0	1	1	0	0	1	1	1	0	1	0	1	1	0	0						
																							
0	+	1	-	1	+	1	0	-	1	0	+	1	0	0	-	1	+	1	0	-	1	0	0

Booth recoding of a multiplier.

Booth Algorithm

$$\begin{array}{r}
 0 \ 1 \ 1 \ 0 \ 1 \quad (+13) \\
 \times 1 \ 1 \ 0 \ 1 \ 0 \quad (-6) \\
 \hline
 \end{array}$$

$$\begin{array}{r}
 \begin{array}{ccccc}
 0 & 0 & 0 & 0 & 0 \\
 1 & 1 & 1 & 1 & 1 \\
 0 & 0 & 0 & 0 & 1 \\
 1 & 1 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0
 \end{array}
 \end{array}
 \begin{array}{c}
 \Rightarrow \\
 \Rightarrow
 \end{array}
 \begin{array}{r}
 \begin{array}{ccccc}
 0 & 1 & 1 & 0 & 1 \\
 0 & -1 & +1 & -1 & 0 \\
 \hline
 0 & 0 & 0 & 0 & 0 \\
 0 & 0 & 1 & 1 & \\
 1 & 0 & 1 & & \\
 1 & 1 & & & \\
 0 & & & &
 \end{array} \\
 \hline
 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \quad (-78)
 \end{array}$$

Booth multiplication with a negative multiplier.

Booth Algorithm

Multiplier		Version of multiplicand selected by bit i
Bit i	Bit $i - 1$	
0	0	0 X M
0	1	+ 1 X M
1	0	- 1 X M
1	1	0 X M

Booth multiplier recoding table.

Booth Algorithm

- Best case – a long string of 1's (skipping over 1s)
- Worst case – 0's and 1's are alternating

Worst-case multiplier

0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
															
+1	-1	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1	+1	-1

Ordinary multiplier

1	1	0	0	0	1	0	1	1	0	1	1	1	1	0	0
															
0	-1	0	0	+1	-1	+1	0	-1	+1	0	0	0	-1	0	0

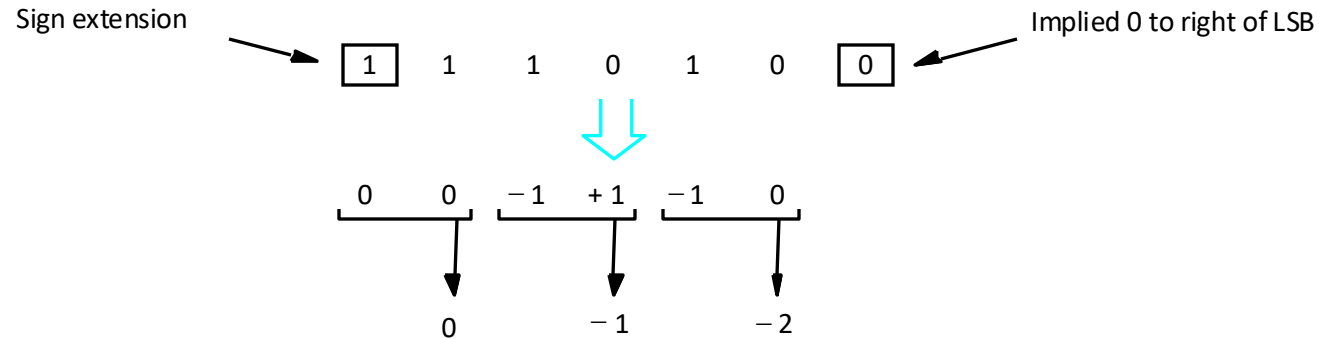
Good multiplier

0	0	0	0	1	1	1	1	1	0	0	0	0	1	1	1
															
0	0	0	+1	0	0	0	0	-1	0	0	0	+1	0	0	-1

Fast Multiplication

Bit-Pair Recoding of Multipliers

- Bit-pair recoding halves the maximum number of summands (versions of the multiplicand).



(a) Example of bit-pair recoding derived from Booth recoding

Bit-Pair Recoding of Multipliers

Multiplier bit-pair		Multiplier bit on the right $i - 1$	Multiplicand selected at position i
$i + 1$	i		
0	0	0	0 X M
0	0	1	+ 1 X M
0	1	0	+ 1 X M
0	1	1	+ 2 X M
1	0	0	- 2 X M
1	0	1	- 1 X M
1	1	0	- 1 X M
1	1	1	0 X M

(b) Table of multiplicand selection decisions

Bit-Pair Recoding of Multipliers

$$\begin{array}{r}
 0 \ 1 \ 1 \ 0 \ 1 \ (+13) \\
 1 \ 1 \ 0 \ 1 \ 0 \ (-6) \\
 \hline
 \end{array}$$



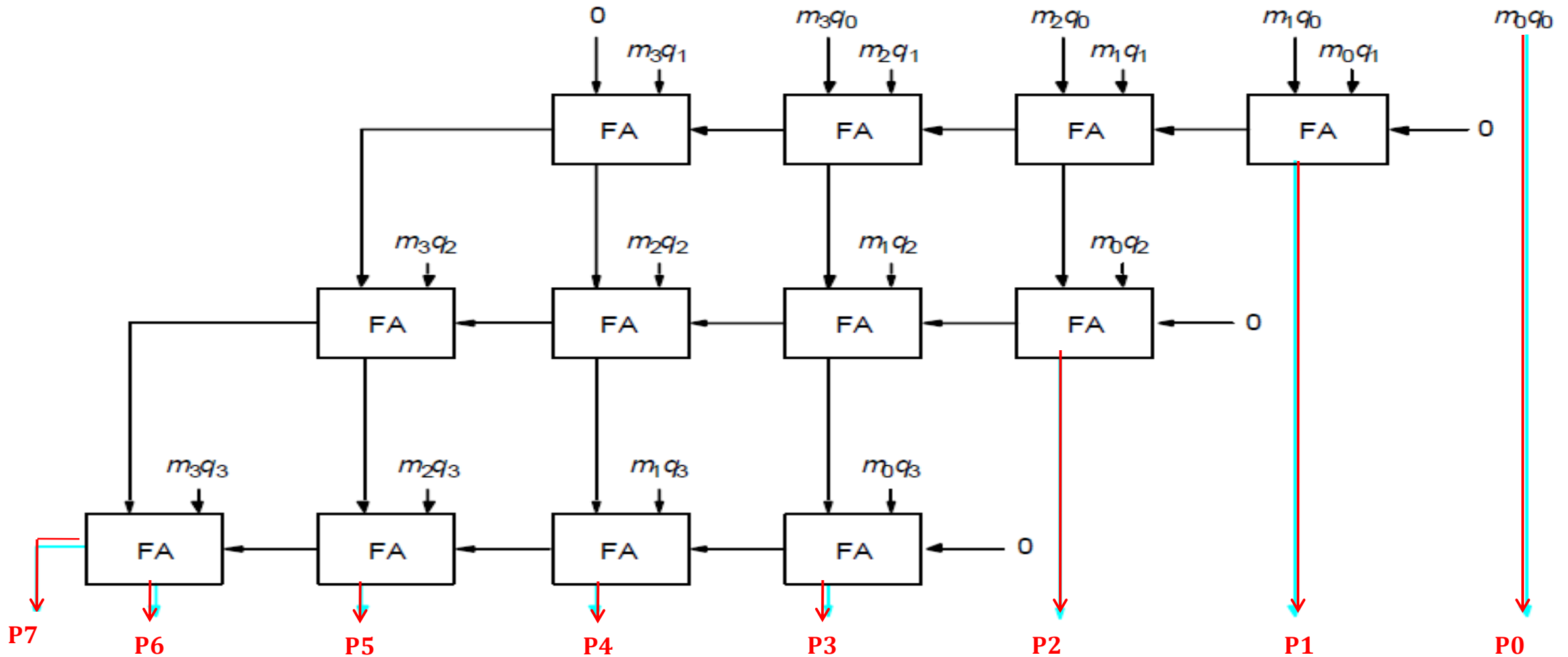
$$\begin{array}{r}
 \begin{array}{ccccc}
 0 & 1 & 1 & 0 & 1 \\
 0 & -1 & +1 & -1 & 0 \\
 \hline
 0 & 0 & 0 & 0 & 0 \\
 1 & 1 & 1 & 1 & 1 \\
 0 & 0 & 0 & 0 & 1 \\
 1 & 1 & 1 & 0 & 0 \\
 0 & 0 & 0 & 0 & 0
 \end{array} \\
 \hline
 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0 \ (-78)
 \end{array}$$



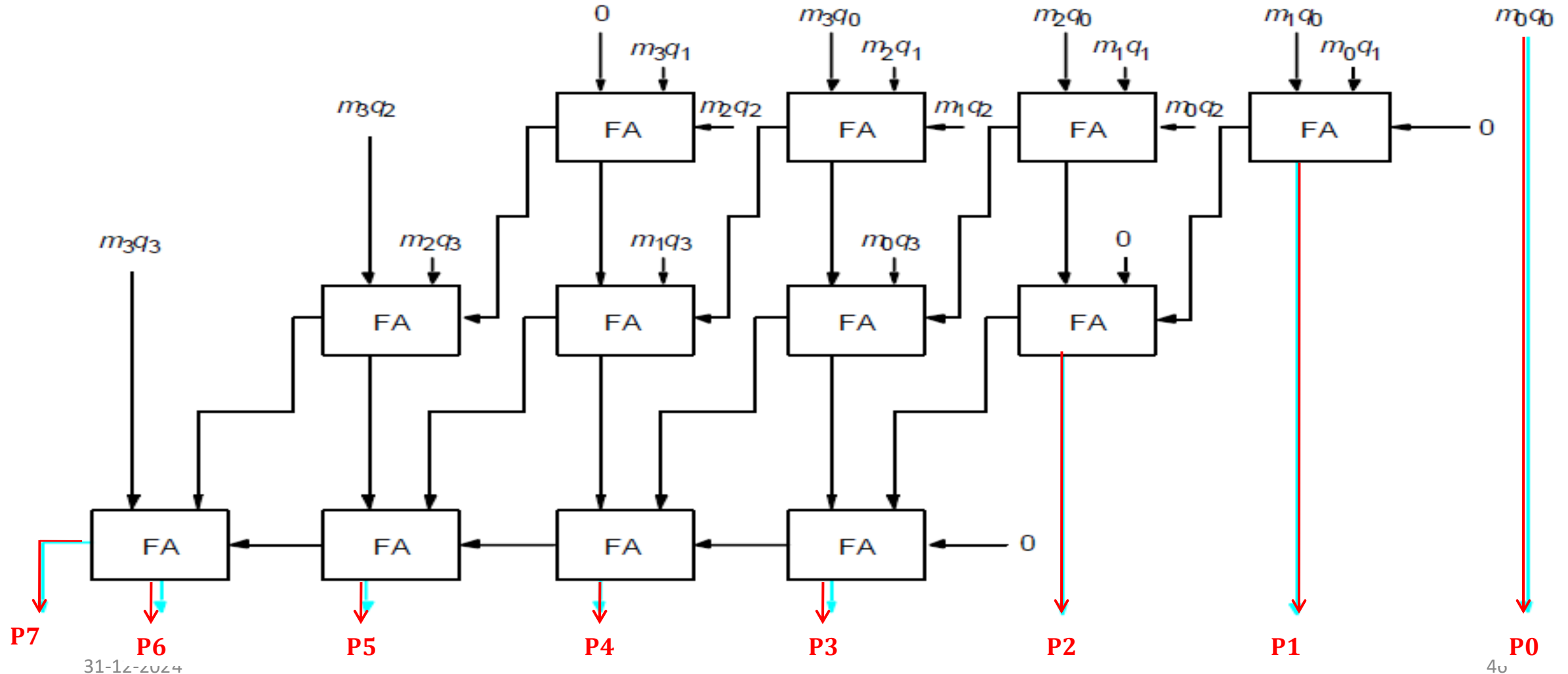
$$\begin{array}{r}
 \begin{array}{ccccc}
 0 & 1 & 1 & 0 & 1 \\
 0 & & -1 & & -2 \\
 \hline
 1 & 1 & 1 & 1 & 1 \\
 1 & 1 & 1 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0
 \end{array} \\
 \hline
 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 0 \ 1 \ 0
 \end{array}$$

Carry-Save Addition of Summands

- CSA speeds up the addition process.



Carry-Save Addition of Summands(Cont.,)



Carry-Save Addition of Summands(Cont.,)

- Consider the addition of many summands, we can:
 - Group the summands in threes and perform carry-save addition on each of these groups in parallel to generate a set of S and C vectors in one full-adder delay
 - Group all of the S and C vectors into threes, and perform carry-save addition on them, generating a further set of S and C vectors in one more full-adder delay
 - Continue with this process until there are only two vectors remaining
 - They can be added in a RCA or CLA to produce the desired product

Carry-Save Addition of Summands

						1	0	1	1	0	1	(45)	M
					x	1	1	1	1	1	1	(63)	Q
						1	0	1	1	0	1	A	
				1		0	1	1	0	1		B	
			1	0		1	1	0	1			C	
				1	0	1	1	0	1			D	
			1	0	1	1	0	1				E	
		1	0	1	1	0	1					F	
1	0	1	1	0	0	0	1	0	0	1	1	(2,835)	Product

Figure 6.17. A multiplication example used to illustrate carry-save addition as shown in Figure 6.18.

Integer Division

Manual Division

$$\begin{array}{r} 21 \\ 13 \overline{) 274} \\ \underline{26} \\ 14 \\ 13 \\ \underline{13} \\ 1 \end{array}$$

$$\begin{array}{r} 10101 \\ 1101 \overline{) 100010010} \\ \underline{1101} \\ 10000 \\ \underline{1101} \\ 1110 \\ \underline{1101} \\ 1 \end{array}$$

Longhand division examples.

Longhand Division Steps

- Position the divisor appropriately with respect to the dividend and performs a subtraction.
- If the remainder is zero or positive, a quotient bit of 1 is determined, the remainder is extended by another bit of the dividend, the divisor is repositioned, and another subtraction is performed.
- If the remainder is negative, a quotient bit of 0 is determined, the dividend is restored by adding back the divisor, and the divisor is repositioned for another subtraction.

Circuit Arrangement

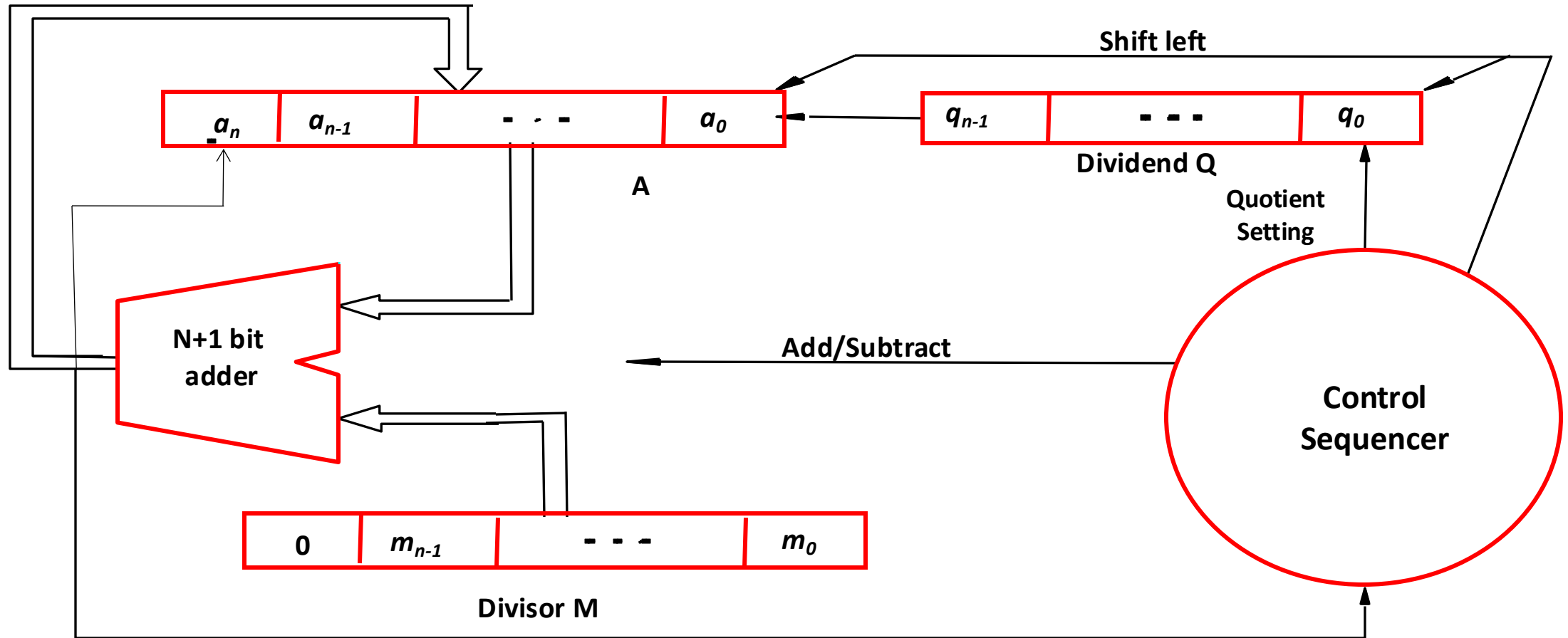


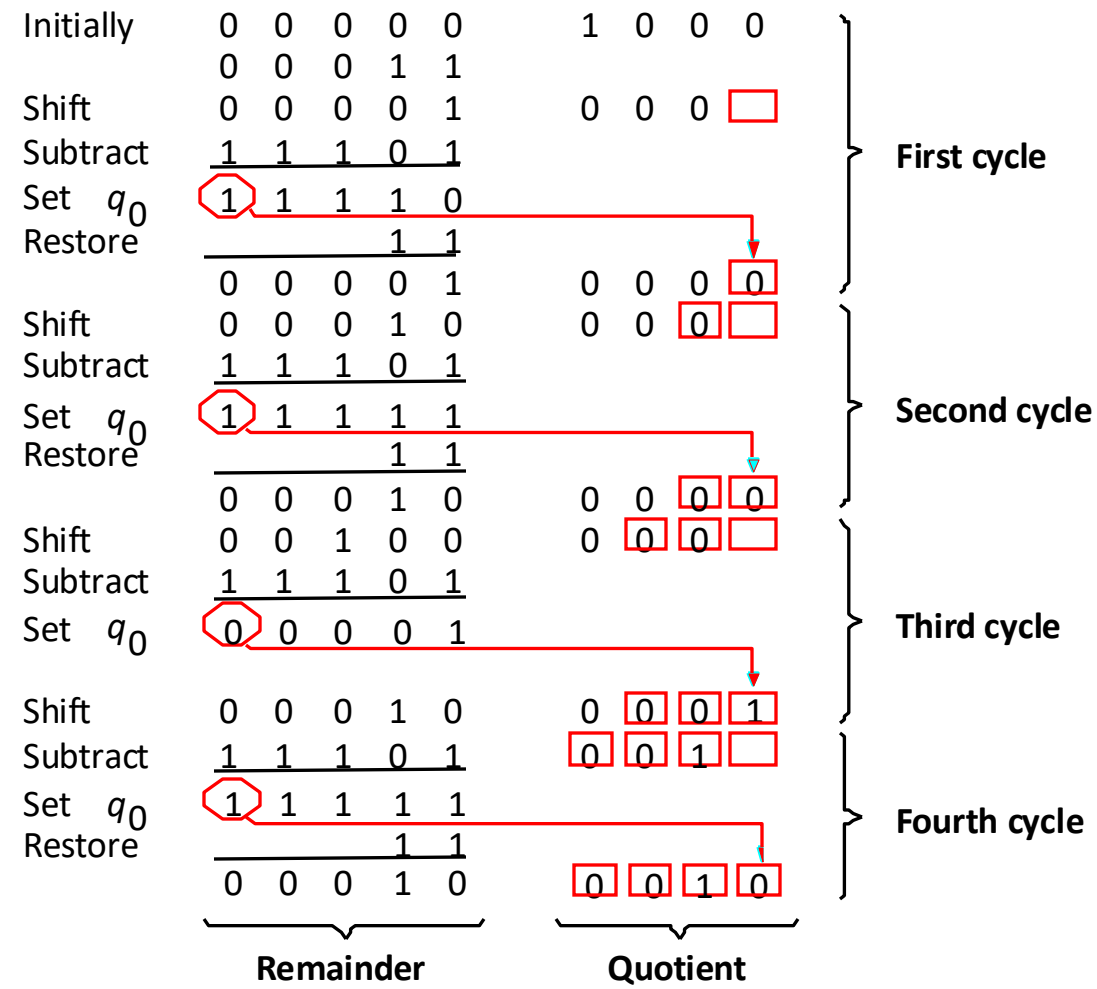
Figure 6.21. Circuit arrangement for binary division.

Restoring Division

- Shift A and Q left one binary position
- Subtract M from A, and place the answer back in A
- If the sign of A is 1, set q_0 to 0 and add M back to A (restore A); otherwise, set q_0 to 1
- Repeat these steps n times

Examples

$$\begin{array}{r}
 11 \overline{) 1000} \\
 \underline{11} \\
 10
 \end{array}$$



Nonrestoring Division

- Avoid the need for restoring A after an unsuccessful subtraction.
- Any idea?
- Step 1: (Repeat n times)
 - If the sign of A is 0, shift A and Q left one bit position and subtract M from A; otherwise, shift A and Q left and add M to A.
 - Now, if the sign of A is 0, set q_0 to 1; otherwise, set q_0 to 0.
- Step2: If the sign of A is 1, add M to A

Examples

Add	$ \begin{array}{r} 1\ 1\ 1\ 1\ 1 \\ 0\ 0\ 0\ 1\ 1 \\ \hline 0\ 0\ 0\ 1\ 0 \end{array} $	}	Restore remainder
	Remainder		

Initially	0	0	0	0	0	1	0	0	0	} First cycle
	0	0	0	1	1					
Shift	0	0	0	0	1	0	0	0		
Subtract	1	1	1	0	1					
Set q_0	1	1	1	1	0	0	0	0	0	} Second cycle
Shift	1	1	1	0	0	0	0	0		
Add	0	0	0	1	1					
Set q_0	1	1	1	1	1	0	0	0	0	} Third cycle
Shift	1	1	1	1	0	0	0	0		
Add	0	0	0	1	1					
Set q_0	0	0	0	0	0	1	0	0	0	} Fourth cycle
Shift	0	0	0	1	0	0	0	1		
Subtract	1	1	1	0	1					
Set q_0	1	1	1	1	1	0	0	1	0	

Quotient

Model Questions

1. Explain with figure the design and working of a 16 bit carry look ahead adder built from 4 bit adders.
2. Explain booth algorithm. Apply booth algorithm to multiply the signed numbers $+13 \times -6$ and $-13 \times +9$.
3. Write circuit arrangement for sequential binary multiplier, explain with example.
4. Differentiate between restoring and non - restoring division. Perform restoring division for the given binary numbers $1000/11$, show all cycles.
5. Design 4 bit carry look ahead logic and explain how it is faster than 4 bit ripple adder.
6. Multiply 14×-8 using booth's algorithm.
7. Explain normalization, excess exponent and special values with respect to IEEE floating point representation.
8. With figure explain circuit arrangements for binary division.

Registers:

A register is a tiny, fast storage memory within the central processing unit (CPU) or the [arithmetic logic unit](#) (ALU) of a computer. Registers are utilized for a variety of functions in handling and controlling instructions and data and play an important role in the operation of a computer's CPU.

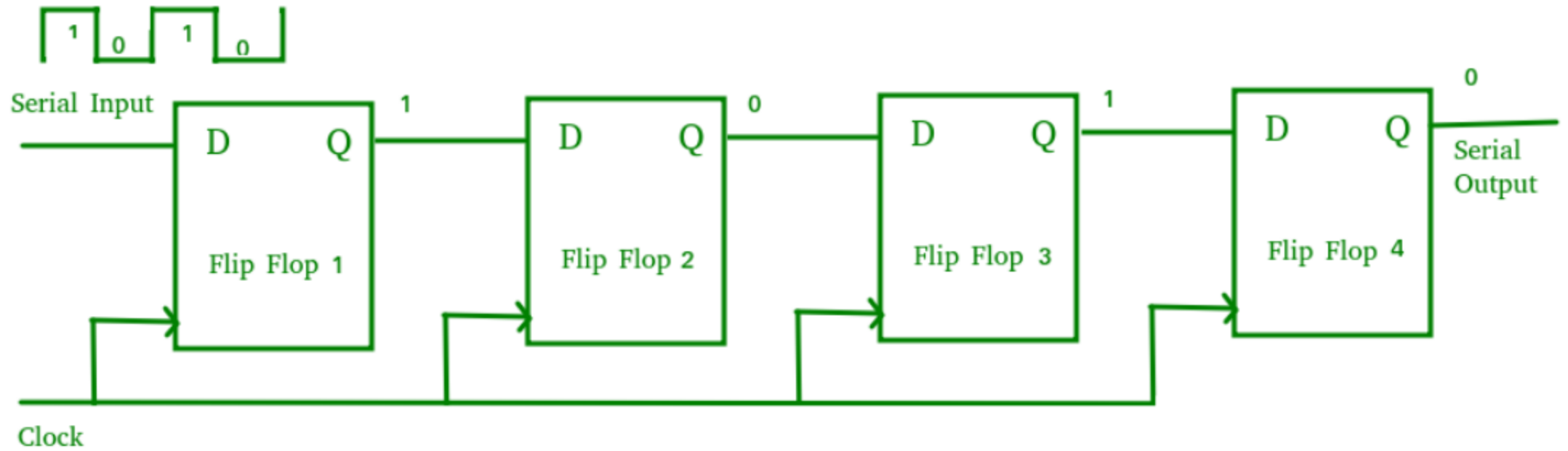
Shift Register is a group of flip flops used to store multiple bits of data. The bits stored in such registers can be made to move within the registers and in/out of the registers by applying clock pulses. An n-bit shift register can be formed by connecting n flip-flops where each flip-flop stores a single bit of data. The registers which will shift the bits to the left are called “Shift left registers”. The registers which will shift the bits to the right are called “Shift right registers”.

Types of Shift Registers

- Serial In Serial Out shift register
- Serial In parallel Out shift register
- Parallel In Serial Out shift register
- Parallel In parallel Out shift register

SISO: Serial In and Serial OUT

The shift register, which allows serial input (one bit after the other through a single data line) and produces a serial output is known as a Serial-In Serial-Out shift register. Since there is only one output, the data leaves the shift register one bit at a time in a serial pattern, thus the name Serial-In Serial-Out Shift Register. The logic circuit given below shows a serial-in serial-out shift register. The circuit consists of four [D flip-flops](#) which are connected in a serial manner. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.

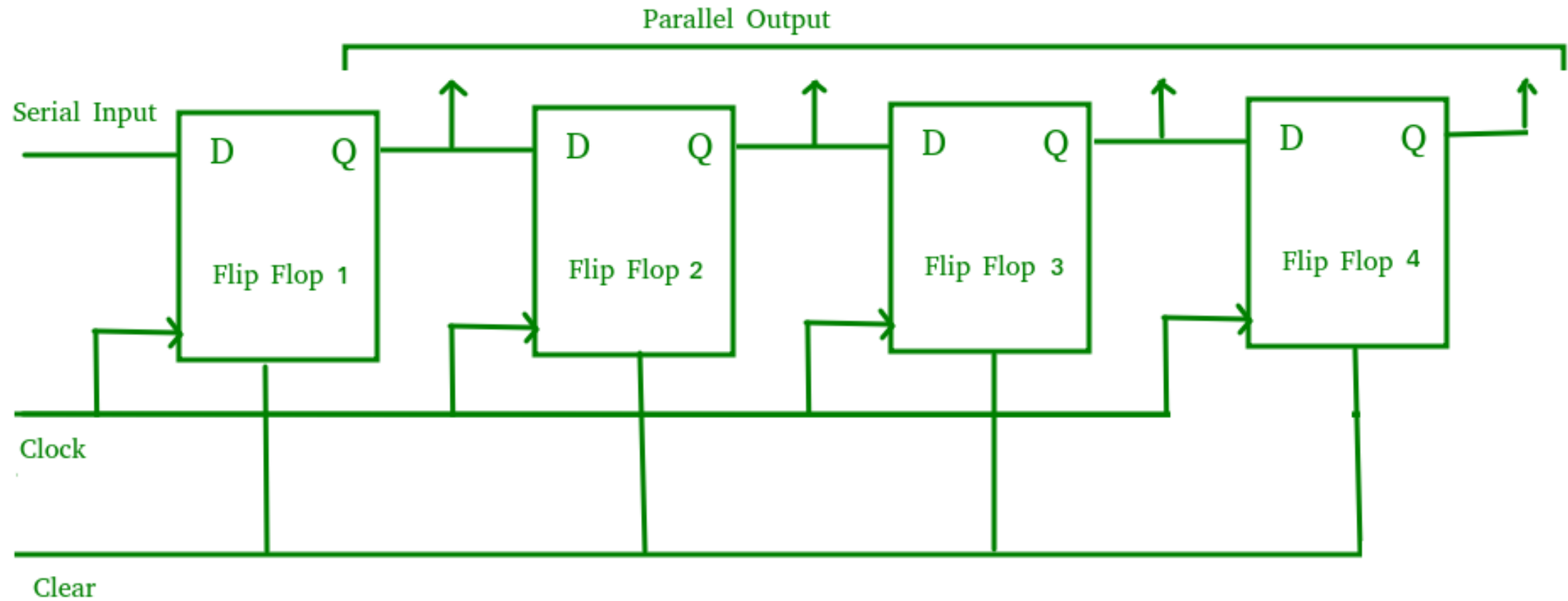


Serial-In Serial-Out Shift Register (SISO)

Serial-In Parallel-Out Shift Register (SIPO)

The shift register, which allows serial input (one bit after the other through a single data line) and produces a parallel output is known as the Serial-In Parallel-Out shift register. The logic circuit given below shows a serial-in-parallel-out shift register. The circuit consists of four D flip-flops which are connected. The clear (CLR) signal is connected in addition to the clock signal to all 4 flip flops in order to RESET them. The output of the first flip-flop is connected to the input of the next flip flop and so on. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.

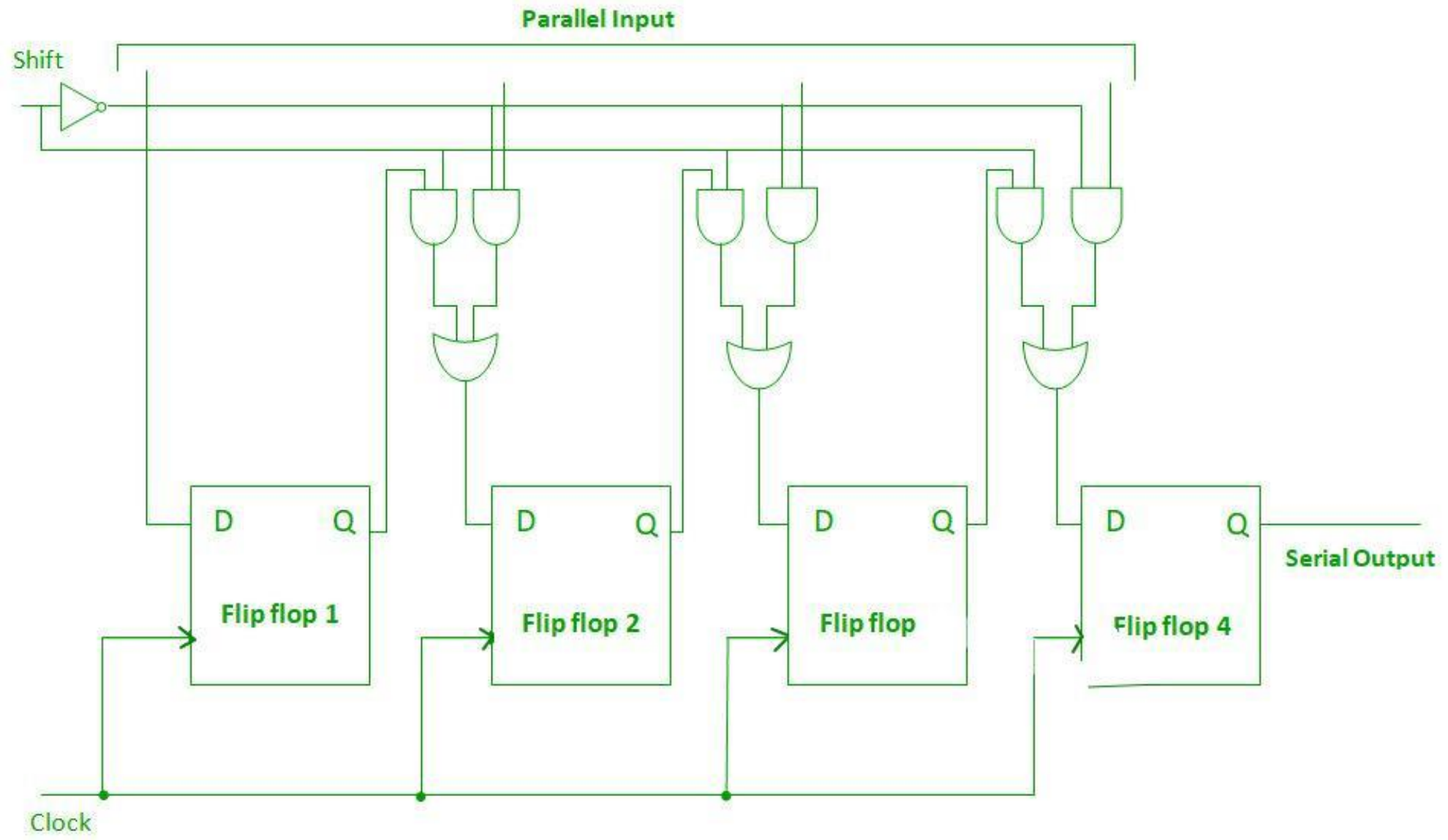
They are used in communication lines where demultiplexing of a data line into several parallel lines is required because the main use of the SIPO register is to convert serial data into parallel data.



Serial-In Parallel-Out Shift Register (SIPO)

Parallel-In Serial-Out Shift Register (PISO)

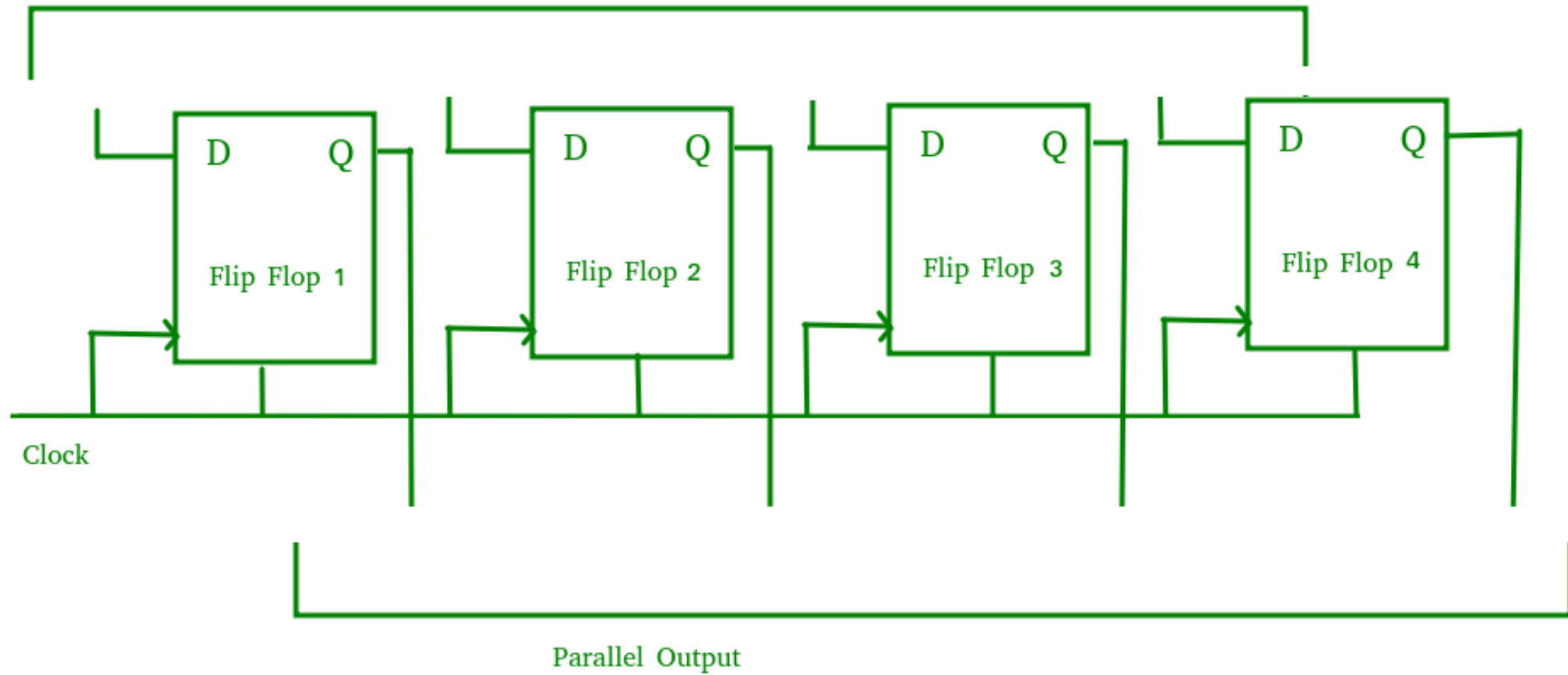
The shift register, which allows parallel input (data is given separately to each flip flop and in a simultaneous manner) and produces a serial output is known as a Parallel-In Serial-Out shift register. The logic circuit given below shows a parallel-in-serial-out shift register. The circuit consists of four D flip-flops which are connected. The clock input is directly connected to all the flip-flops but the input data is connected individually to each flip-flop through a [multiplexer](#) at the input of every flip-flop. The output of the previous flip-flop and parallel data input are connected to the input of the MUX and the output of MUX is connected to the next flip-flop. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.



Parallel-In Serial-Out Shift Register (PISO)

Parallel-In Parallel-Out Shift Register (PIPO)

The shift register, which allows parallel input (data is given separately to each flip flop and in a simultaneous manner) and also produces a parallel output is known as Parallel-In parallel-Out shift register. The logic circuit given below shows a parallel-in-parallel-out shift register. The circuit consists of four D flip-flops which are connected. The clear (CLR) signal and clock signals are connected to all 4 flip-flops. In this type of register, there are no interconnections between the individual flip-flops since no serial shifting of the data is required. Data is given as input separately for each flip flop and in the same way, output is also collected individually from each flip flop.



Parallel-In Parallel-Out Shift Register (PIPO)